生物统计和生物信息引论

Lecture 10

深度学习基础

王健桥
jianqw@mail.tsinghua.edu.cn

清華大學 统计与数据科学系
Department of Statistics and Data Science, Tsinghua University

Motivation: Why AI in Biomedicine?

- 生物医学数据规模快速增长：医学影像、基因组、电子病历、可穿戴设备、实验数据
- 难以处理的多模态、非结构化数据
- 什么是多模态?
- 什么是非结构化数据?

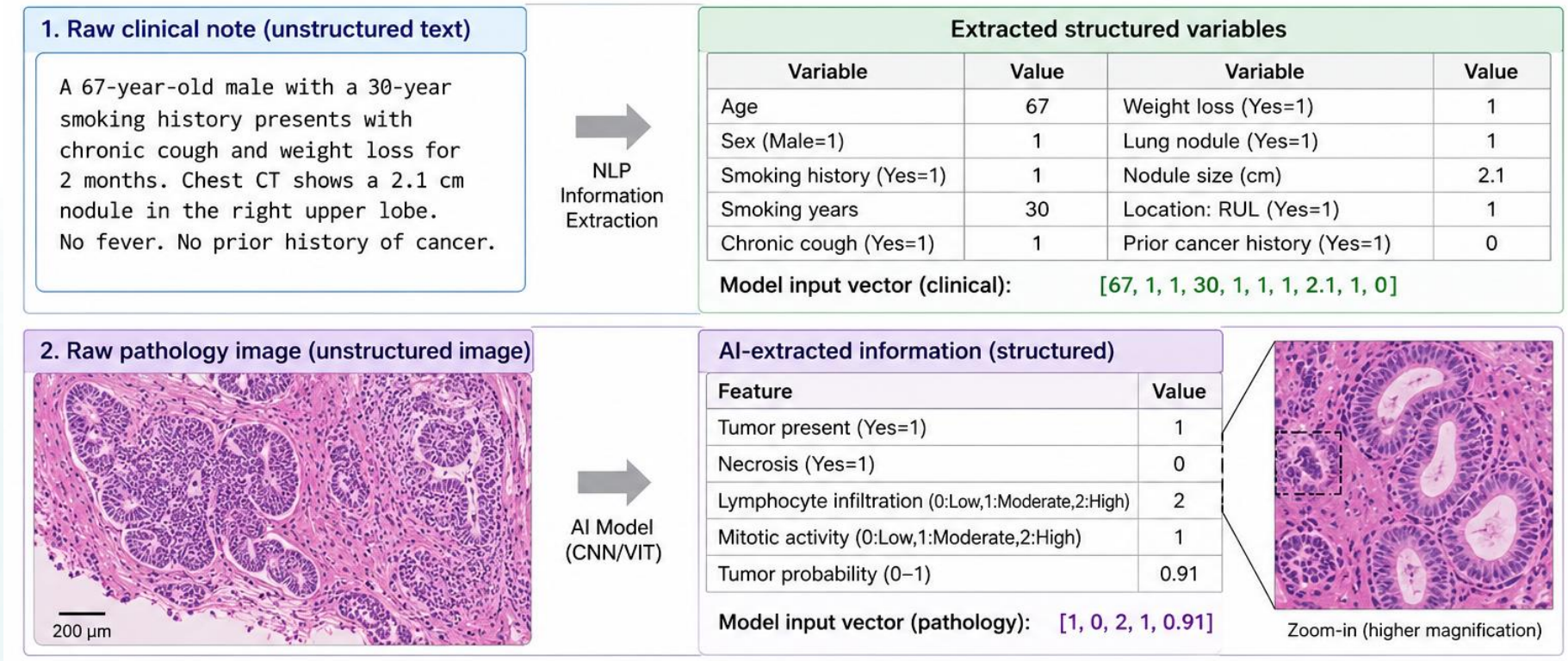
Example: Multimodal AI for disease Prognosis

- Input:
  - Imaging: CT / pathology images
  - Omics: Protein, metabolites
  - Clinical records: Age, stage, treatment
  - Clinical notes: Pathology reports
- Output:
  - Survival prediction
  - Treatment response
  - Disease subtype

Structured data & Unstructured data

Structured data are easier to analyze, but may lose important information.

Raw biomedical data contain richer information, but also more noise and redundancy.



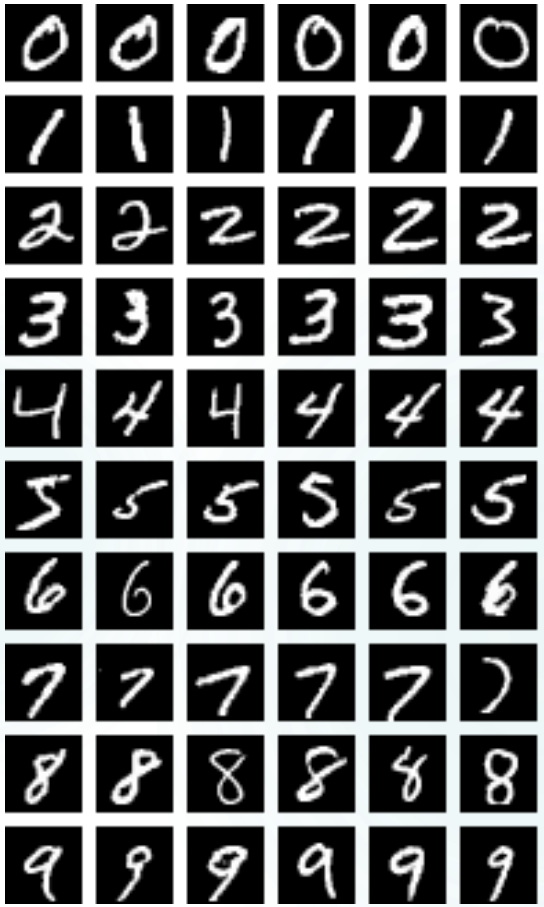
GPT image

Outline

- Task and Data
- Binary Logistic Regression
- Multi-class Logistic Regression
- Logistic Regression with MLP

Task and Data

MNIST dataset



- MNIST is a benchmark dataset of handwritten digits from 0 to 9.
- It contains 70,000 grayscale images, each with size 28 × 28 pixels.
- The task is a **10-class classification** problem: given an image, predict which digit it represents.
- MNIST is widely used to introduce **machine learning, neural networks, and image classification** because it is simple, clean, and easy to visualize.
- Although it is relatively easy for modern models, it remains a useful starting point for understanding how models learn from image data.

Task: 10-class classification



Model

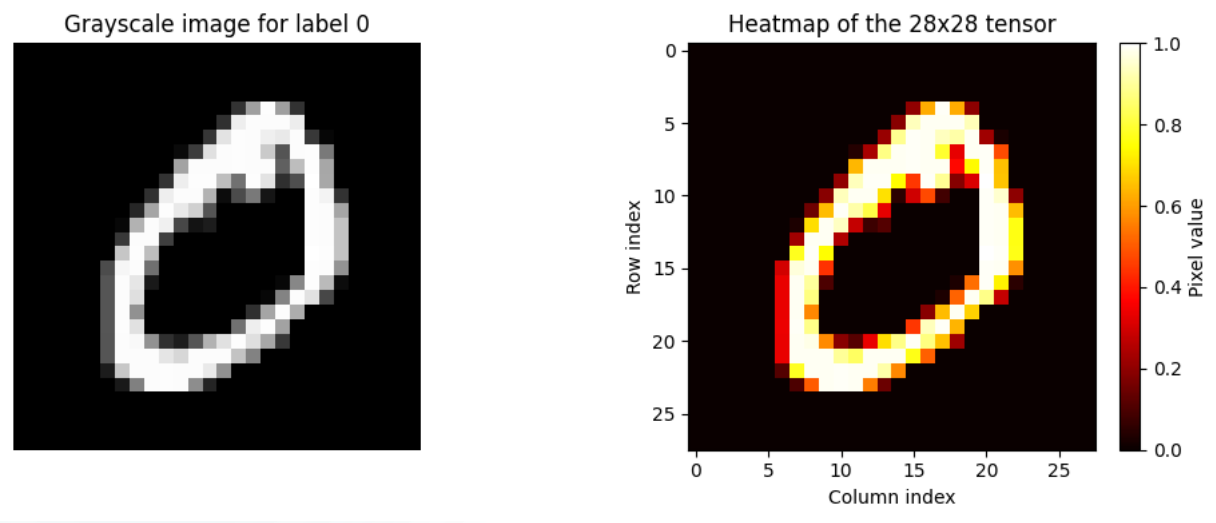
0, 1, …, 9

Input x: figures

Output y: labels

What does data look like

- They are just 28*28 matrix, each element is between 0 and 1
- A mapping $[0,1]^{28\times 28} \to \{0,1,\ldots,9\}$
- However, traditional methods cannot handle non-structural data.
- We can flatten the figure as a 784-dim vector



代码：dl_intro/01_mnist_dataset_overview.ipynb

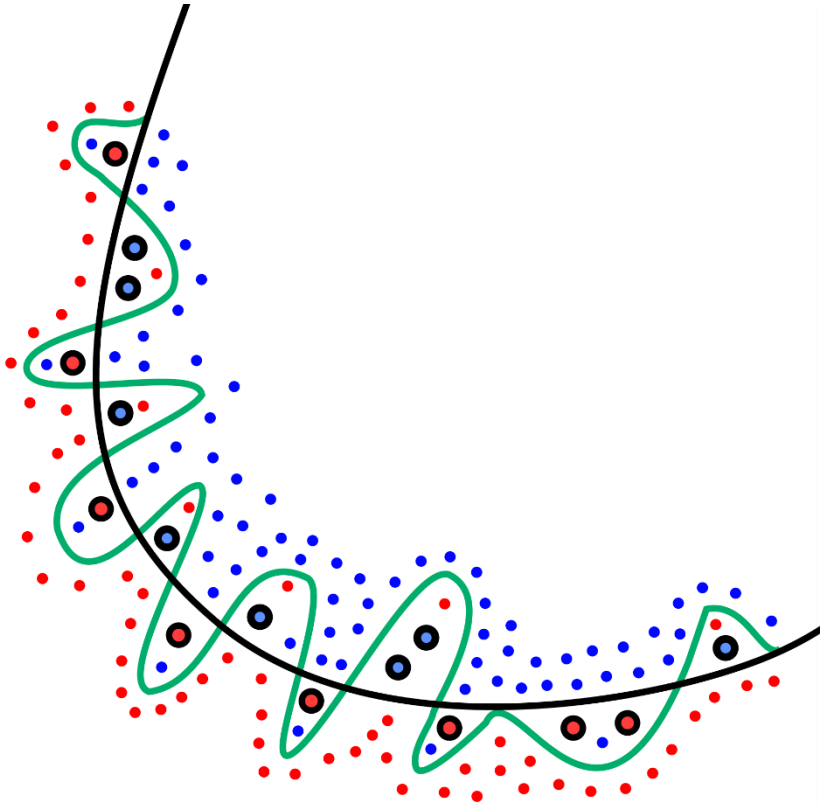
Training/Testing set split

- We usually split all available data into training and testing set.
- Training data is for model training
- Testing data is for model performance testing
- We don’t care if the model “remember everything”, but if the model can find rules and perform well on unseen data.

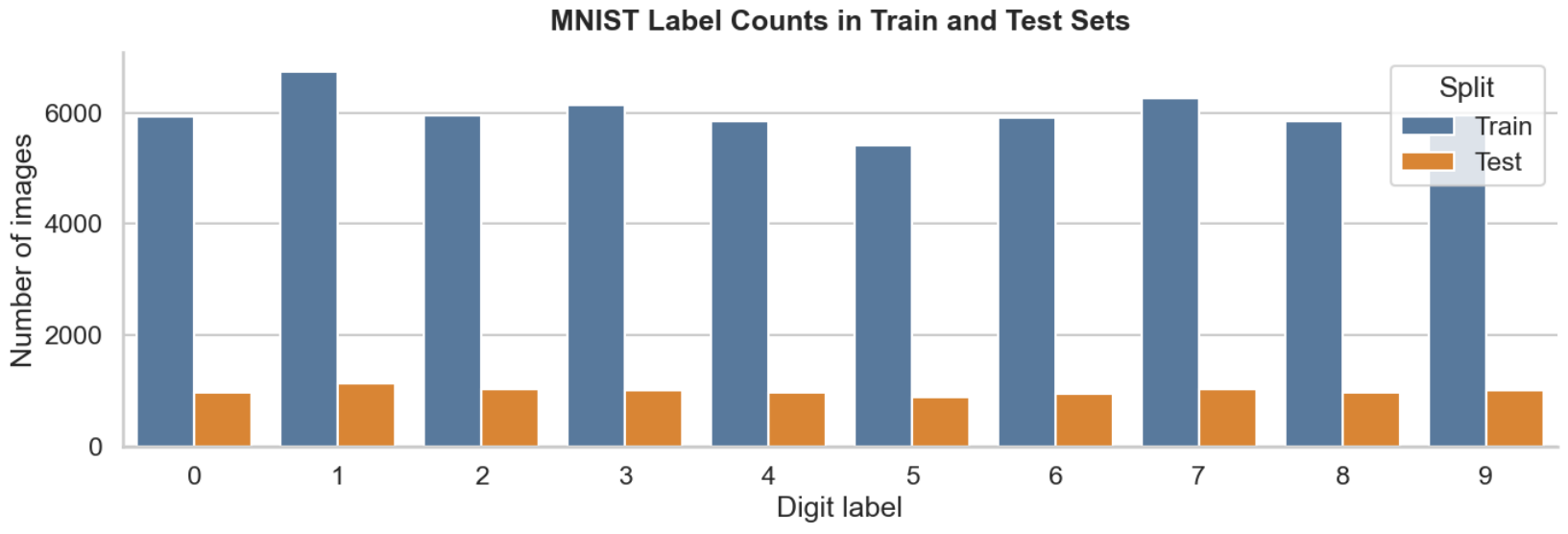


Overfitting

- Overfitting = doing very well on training data, but poorly on new data
- The model learns not only the pattern, but also noise and randomness
- It may “memorize” the training examples instead of learning general rules
- As a result, training error is low, while test error stays high
- Our goal is not to fit the training set perfectly, but to generalize to unseen data



Split in MNIST



- Train set for learning, test set for evaluation
- Keep class proportions similar across splits
- Balanced splits make performance comparison more reliable

Metric

- Accuracy measures how often the model is correct

$$\text{Accuracy} = \frac{\text{correct predictions}}{\text{total predictions}}$$

- Example: 9 correct out of 10 → accuracy = 90%
- We usually care more about **test accuracy** than train accuracy

- Other: recall （漏检少不少）, precision （预测准不准）, F1 …

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \qquad \text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

	Predicted Positive	Predicted Negative
Actual Positive	**TP**: True Positive — correctly predicted positive	**FN**: False Negative — positive case missed
Actual Negative	**FP**: False Positive — incorrectly predicted positive	**TN**: True Negative — correctly predicted negative

Binary Logistic Regression

Recall: Binary Logistic Regression

- We observe input – label pairs

$$(x_i, y_i), \qquad y_i \in \{0, 1\}$$

- Goal: model the probability of the positive class

$$\pi_i = P(Y_i = 1 \mid x_i)$$

- Direct linear regression is not suitable, because probabilities must satisfy

$$0 \leq \pi_i \leq 1$$

- Logistic regression models the **log-odds** as a linear function:

$$\log \frac{\pi_i}{1 - \pi_i} = \eta_i = \beta^\top x_i + b$$

- Solving for π_i, we get the sigmoid form:

$$\pi_i = \frac{e^{\eta_i}}{1 + e^{\eta_i}} = \frac{1}{1 + e^{-\eta_i}} = \sigma(\eta_i)$$

Recall: Binary Logistic Regression

- Assume the label follows a Bernoulli distribution:

$$Y_i \mid x_i \sim \text{Bernoulli}(\pi_i)$$

- Therefore, the probability of observing y_i is:

$$P(Y_i = y_i \mid x_i) = \pi_i^{y_i}(1-\pi_i)^{1-y_i}$$

- For independent samples, the likelihood is:

$$L(\beta, b) = \prod_{i=1}^{n} \pi_i^{y_i}(1-\pi_i)^{1-y_i}$$

- The log-likelihood is: $\ell(\beta, b) = \sum_{i=1}^{n} [y_i \log \pi_i + (1-y_i)\log(1-\pi_i)]$
- Maximum likelihood estimation: $(\hat{\beta}, \hat{b}) = \arg\max_{\beta, b} \ell(\beta, b)$
- Equivalently, minimize the negative log-likelihood:

$$\mathcal{L}(\beta, b) = -\frac{1}{n}\sum_{i=1}^{n} [y_i \log \pi_i + (1-y_i)\log(1-\pi_i)]$$

Cross-Entropy

$$\mathcal{L}(\beta, b) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log \pi_i + (1-y_i)\log(1-\pi_i)\right]$$

- It is also called cross-entropy loss
- Cross entropy loss is used to compare two probability distributions
- General form:

$$H(y, p) = -\sum_{k} y_k \log(p_k)$$

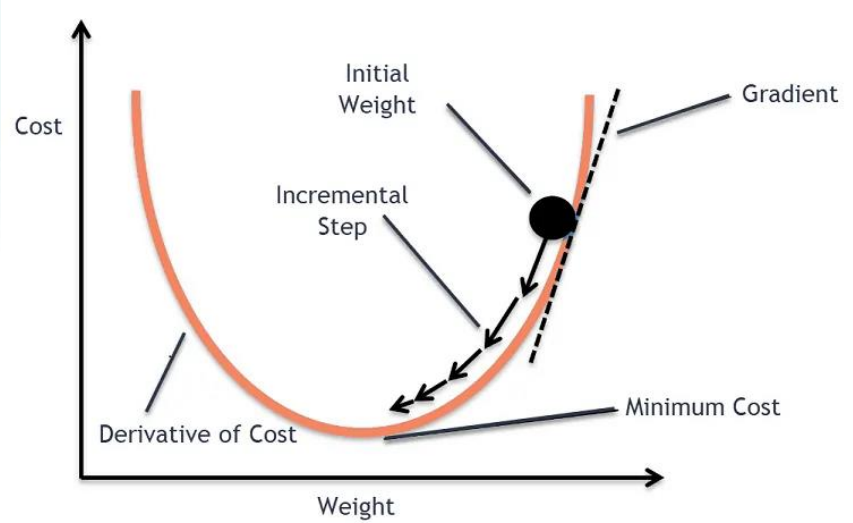
- Here, y_k means the true probability of class k, and p_k means the model-predicted probability of class k.
- In classification problem, we use CE loss, and is equivalent to MLE assuming Multinomial Distribution

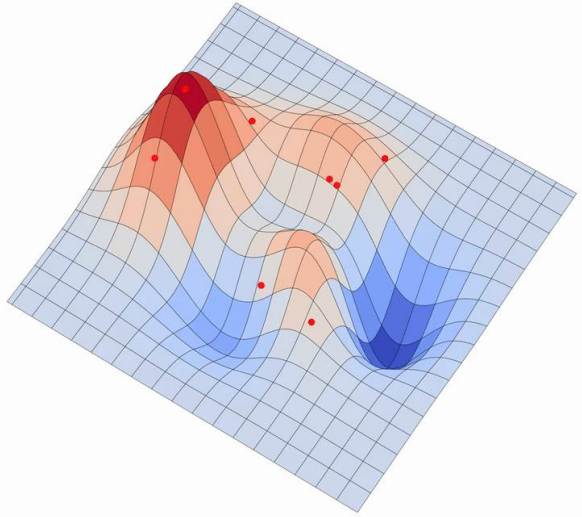
Optimization: Gradient Descent

- There is no close-form solution for LR, we need to start from initial parameters and improve them step by step.
- The gradient tells us the direction in which the loss increases fastest.
- Therefore, we move in the opposite direction:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_\theta \mathcal{L}(\theta^{(t)})$$

- θ can represent all parameters. Here $\theta = (\beta, b)$, and α is learning rate





Gradient for Binary Logistic Regression

- For one sample:

$$\mathcal{L}_i = -\left[y_i \log \pi_i + (1 - y_i) \log(1 - \pi_i)\right]$$

$$\pi_i = \sigma(\eta_i) = \frac{1}{1 + e^{-\eta_i}}$$

- By chain rule:

$$\frac{\partial \mathcal{L}_i}{\partial \eta_i} = \frac{\partial \mathcal{L}_i}{\partial \pi_i} \cdot \frac{\partial \pi_i}{\partial \eta_i}$$

- Firstly,

$$\frac{\partial \mathcal{L}_i}{\partial \pi_i} = -\frac{y_i}{\pi_i} + \frac{1 - y_i}{1 - \pi_i}$$

- Second,

$$\frac{\partial \pi_i}{\partial \eta_i} = \pi_i(1 - \pi_i)$$

- Therefore,

$$\frac{\partial \mathcal{L}_i}{\partial \eta_i} = \left(-\frac{y_i}{\pi_i} + \frac{1 - y_i}{1 - \pi_i}\right) \pi_i(1 - \pi_i) = -y_i(1 - \pi_i) + (1 - y_i)\pi_i = \pi_i - y_i$$

Gradient for Binary Logistic Regression

- Then since

$$\eta_i = \beta^\top x_i + b$$

- we have

$$\frac{\partial \mathcal{L}_i}{\partial \beta} = (\pi_i - y_i)x_i$$

$$\frac{\partial \mathcal{L}_i}{\partial b} = \pi_i - y_i$$

- If $\pi_i > y_i$:model overpredicts the positive class.
- If $\pi_i < y_i$:model underpredicts the positive class.
- The error signal is simply:

$$\pi_i - y_i$$

Gradient Descent

- For all samples, the average loss is: $\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n}\mathcal{L}_i$
- Therefore,

$$\nabla_\beta \mathcal{L} = \frac{1}{n}\sum_{i=1}^{n}(\pi_i - y_i)x_i$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n}\sum_{i=1}^{n}(\pi_i - y_i)$$

- Gradient descent update:

$$\beta \leftarrow \beta - \alpha\frac{1}{n}\sum_{i=1}^{n}(\pi_i - y_i)x_i$$

$$b \leftarrow b - \alpha\frac{1}{n}\sum_{i=1}^{n}(\pi_i - y_i)$$

Gradient Descent

- Matrix form

$$X \in \mathbb{R}^{n \times d}, \qquad \eta = X\beta + b\mathbf{1}, \qquad \pi = \sigma(\eta)$$

$$\nabla_\beta \mathcal{L} = \frac{1}{n} X^\top (\pi - y)$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n} \mathbf{1}^\top (\pi - y)$$

- In code implementation, we usually use matrix form rather than for loop to fully utilize parallelization.

- **Forward**: compute η, π, and loss.
- **Backward**: compute gradients such as $\nabla_\beta \mathcal{L}$.
- **Update**: move parameters in the negative gradient direction.

Stochastic Gradient Descent

- Full-batch gradient descent uses all n samples for every update:

$$\mathcal{L}(\beta, b) = \frac{1}{n}\sum_{i=1}^{n}\mathcal{L}_i, \qquad \nabla_\beta \mathcal{L} = \frac{1}{n}X^\top(\pi - y)$$

- This is mathematically clean, but expensive when the dataset is large
- Mini-batch SGD randomly selects a small batch $B \subset \{1, \ldots, n\}$ and computes the loss only on this batch:

$$\mathcal{L}_B(\beta, b) = \frac{1}{|B|}\sum_{i\in B}\mathcal{L}_i$$

- The mini-batch gradients are: $\nabla_\beta \mathcal{L}_B = \frac{1}{|B|}X_B^\top(\pi_B - y_B), \qquad \frac{\partial \mathcal{L}_B}{\partial b} = \frac{1}{|B|}\mathbf{1}^\top(\pi_B - y_B)$
- Parameter update:

$$\beta \leftarrow \beta - \alpha\nabla_\beta \mathcal{L}_B, \qquad b \leftarrow b - \alpha\frac{\partial \mathcal{L}_B}{\partial b}$$

- Key idea: the mini-batch gradient is a **noisy** but much **cheaper** approximation of the full gradient.

Binary Logistic Regression: Pseudocode

Goal: estimate (β, b) by minimizing the negative log-likelihood.

1 Initialize parameters
Choose initial values for β and b (e.g., zeros or small random values).

$$\beta^{(0)},\ b^{(0)}$$

2 Forward computation
For each sample $i = 1, \ldots, n$

$$\eta_i = \beta^\top x_i + b$$
$$\pi_i = \sigma(\eta_i) = \frac{1}{1 + e^{-\eta_i}}$$

3 Compute loss
Evaluate the average negative log-likelihood

$$\mathcal{L}(\beta, b) = -\frac{1}{n} \sum_{i=1}^{n} [y_i \log \pi_i + (1 - y_i) \log(1 - \pi_i)]$$

4 Compute gradients
Differentiate the loss with respect to β and b

$$\frac{\partial \mathcal{L}}{\partial \beta} = \frac{1}{n} \sum_{i=1}^{n} (\pi_i - y_i) x_i \qquad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n} \sum_{i=1}^{n} (\pi_i - y_i)$$

5 Update parameters
Use gradient descent with learning rate α

$$\beta \leftarrow \beta - \alpha \frac{\partial \mathcal{L}}{\partial \beta}$$
$$b \leftarrow b - \alpha \frac{\partial \mathcal{L}}{\partial b}$$

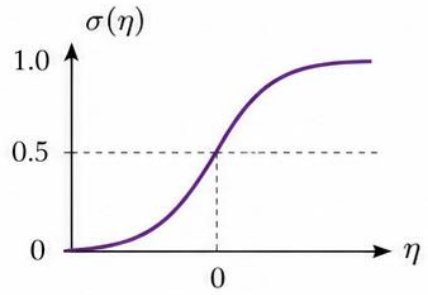
6 Check stopping criterion
Stop if the loss changes very little or the maximum number of iterations is reached; otherwise repeat from Step 2.

Prediction

For a new input x_{new}:

$$\eta_{\text{new}} = \beta^\top x_{\text{new}} + b$$
$$\pi_{\text{new}} = \sigma(\eta_{\text{new}})$$

Predict class 1 if $\pi_{\text{new}} \geq 0.5$, otherwise class 0.



Notation x_i : feature vector y_i : label π_i : probability of class 1 η_i : linear predictor α : learning rate

Binary Logistic Regression: Modeling Relative Odds

- For binary classification, there are two probabilities:

$$p_0 = P(Y = 0 \mid x), \qquad p_1 = P(Y = 1 \mid x)$$

- Since there are only two classes: $p_0 + p_1 = 1$
- Logistic regression models the log-odds of class 1 against class 0:

$$\log \frac{p_1}{p_0} = \eta = \beta^\top x + b$$

- Equivalently,

$$\frac{p_1}{p_0} = e^\eta$$

- Interpretation: η is a **relative score** for class 1 compared with class 0.
- Since $p_0 + p_1 = 1$,

$$p_1 = \frac{e^\eta}{1 + e^\eta} = \frac{1}{1 + e^{-\eta}} = \sigma(\eta)$$

Two-class normalization

- Set class 0 as the baseline:

$$s_0 = 0, \qquad s_1 = \eta$$

- Then the probabilities can be written as:

$$p_0 = \frac{e^{s_0}}{e^{s_0} + e^{s_1}}$$

$$p_1 = \frac{e^{s_1}}{e^{s_0} + e^{s_1}}$$

- Since $s_0 = 0$ and $s_1 = \eta$,

$$p_1 = \frac{e^{\eta}}{1 + e^{\eta}} = \sigma(\eta)$$

- Therefore, binary logistic regression can be viewed as:
  - assign a score to each class;
  - exponentiate the scores;
  - normalize them so probabilities sum to 1.

Multi-class Logistic Regression

From Binary Logistic to Multiclass LR

- For K-class classification, choose class 0 as the baseline.
- For each other class $k = 1, \ldots, K-1$, model its log-odds against class 0:

$$\log \frac{p_k}{p_0} = \eta_k = w_k^\top x + b_k \qquad p_k = p_0 e^{\eta_k}$$

- Since all probabilities must sum to 1: $p_0 + p_1 + \cdots + p_{K-1} = 1$

$$p_0 \left(1 + \sum_{j=1}^{K-1} e^{\eta_j}\right) = 1$$

- Therefore,

$$p_0 = \frac{1}{1 + \sum_{j=1}^{K-1} e^{\eta_j}} \qquad p_k = \frac{e^{\eta_k}}{1 + \sum_{j=1}^{K-1} e^{\eta_j}}, \qquad k = 1, \ldots, K-1$$

- This is **multiclass logistic regression**. And $\mathrm{softmax}(\boldsymbol{z})_i = \dfrac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$

Softmax

- Softmax function turns arbitrary inputs(logits) into probabilities
- Logits are not probabilities:
  - they can be negative;
  - they can be larger than 1;
  - they do not need to sum to 1.

$$\text{softmax}(\boldsymbol{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

- Property:

$$\text{softmax}(s) = \text{softmax}(s + c\mathbf{1})$$

- Softmax: soft + max.
- Hard max: argmax just choose the largest one.
- For $z = [2.0,\ 1.0,\ 0.1]$, argmax gives $[1,\ 0,\ 0]$, which is a hard decision
- But softmax gives $p \approx [0.66,\ 0.24,\ 0.10]$. The max logit still correspond to max probability, but also keep non-zero prob for other classes. So, it's a soft version of max.

Code: dl_intro/02_softmax_cross_entropy.ipynb

Identifiable Note

- In neural networks, rather than chooses one baseline class, e.g. class 0, we usually predict K logits:

$$s_0, s_1, \ldots, s_{K-1}$$

$$p_k = \frac{\exp(s_k)}{\sum_{j=0}^{K-1} \exp(s_j)}$$

- It's equivalent, since

$$\frac{\exp(s_k)}{\sum_{j=0}^{K-1} \exp(s_j)} = \frac{\exp(s_k - s_0)}{1 + \sum_{j=1}^{K-1} \exp(s_j - s_0)}$$

- Therefore, the baseline formulation is recovered by setting:

$$\eta_k = s_k - s_0$$

Multi-class Logistic Regression

- **Task**: predict a label $\{0, 1, \ldots, 9\}$ from an input feature vector $x \in \mathbb{R}^{784}$
- **Model:** for each class k, compute a linear score $s_k = w_k^\top x + b_k$.
- These 10 scores are converted into probabilities using the softmax function:

$$P(y = k \mid x) = \frac{e^{s_k}}{\sum_{j=0}^{9} e^{s_j}}.$$

- The predicted label is the class with the **highest probability**.
- Training is done by minimizing cross-entropy loss over all classes.

Multinomial Likelihood

- For one image x_i ,the model predicts a probability vector:

$$p_i = (p_{i0}, p_{i1}, \ldots, p_{i9})$$

- Represent the true label as a one-hot vector:

$$\boldsymbol{y}_i = (y_{i0}, y_{i1}, \ldots, y_{i9}) \quad y_{ik} = \begin{cases} 1, & \text{if the true class is } k, \\ 0, & \text{otherwise.} \end{cases}$$

- The probability of observing this label is (one-trial multinomial):

$$P(Y_i = y_i \mid x_i) = \prod_{k=0}^{9} p_{ik}^{y_{ik}}$$

- For independent samples, the likelihood is: $L(W, b) = \prod_{i=1}^{n} \prod_{k=0}^{9} p_{ik}^{y_{ik}}$
- The log-likelihood is: $\ell(W, b) = \sum_{i=1}^{n} \sum_{k=0}^{9} y_{ik} \log p_{ik}$
- Therefore, minimizing the negative log-likelihood gives cross-entropy loss:

$$\mathcal{L}(W, b) = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=0}^{9} y_{ik} \log p_{ik}$$

Cross-Entropy: one-hot case

- In our case, each image has one correct label, we present the true label as a one-hot vector. (categorize)
  - E.g. if true digit is 3, then

$$y = (0, 0, 0, 1, 0, 0, 0, 0, 0, 0)$$

- Model prediction after softmax:

$$p = (p_0, p_1, p_2, p_3, \ldots, p_9)$$

- Calculate cross-entropy

$$H(y, p) = -\sum_{k=0}^{9} y_k \log(p_k)$$

- Because only the correct class has $y_k = 1$, the loss becomes:

$$H(y, p) = -\log(p_{\text{correct}})$$

Optimization

Forward Process: $W, b \longrightarrow s \longrightarrow p \longrightarrow L$

$$x \in \mathbb{R}^{784}, \qquad W \in \mathbb{R}^{10\times 784}, \qquad b \in \mathbb{R}^{10}$$

$$s = Wx + b$$

$$p_k = \frac{e^{s_k}}{\sum_{j=0}^{9} e^{s_j}}$$

$$L = -\sum_{k=0}^{9} y_k \log p_k$$

Backward Process: $\dfrac{\partial L}{\partial W} = \dfrac{\partial L}{\partial s}\dfrac{\partial s}{\partial W}$

$$\frac{\partial L}{\partial s_k} = p_k - y_k$$

$$\frac{\partial L}{\partial W} = (p-y)x^\top \qquad \frac{\partial L}{\partial b} = p - y$$

Update

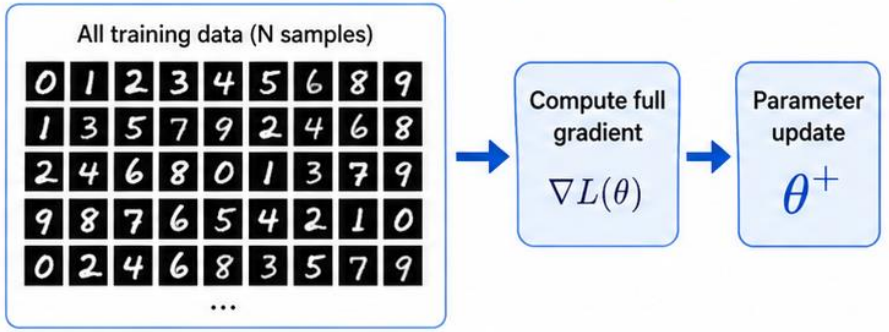
$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

$$b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

GD and SGD

Full Gradient Descent

Use all N samples to compute one update



$$\nabla_\theta L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta L_i(\theta)$$

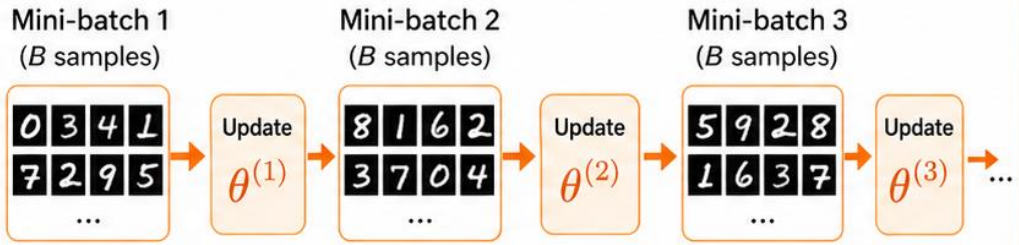
$$\theta \leftarrow \theta - \eta \nabla_\theta L(\theta)$$

Stable / accurate gradient

Slow for large datasets

Mini-batch SGD

Use one sample or a small batch to estimate the gradient



$$\nabla_\theta L_B(\theta) = \frac{1}{B} \sum_{i \in B} \nabla_\theta L_i(\theta)$$

$$\theta \leftarrow \theta - \eta \nabla_\theta L_B(\theta)$$

Much faster updates

Noisier gradient

Standard in deep learning

Full GD: one accurate update per epoch

Mini-batch SGD: many fast updates per epoch

Use SGD: Pytorch Autograd

- Computing gradients by hand is tedious for complex models.
- PyTorch builds a **computation graph** during the forward pass.
- loss.backward() applies the chain rule automatically.
- Gradients are stored in each parameter’s .grad.
- optimizer.step() uses these gradients to update parameters.
- Autograd = automatic chain rule on the computation graph

$$x \to \text{model}(W, b) \to s \to L$$

Computation Graph(Forward)

$$L \to \frac{\partial L}{\partial s} \to \frac{\partial L}{\partial W}, \frac{\partial L}{\partial b}$$

Backward propagation

```
X = images.view(images.shape[0], -1)
optimizer.zero_grad()
# Clear old gradient
logits = model(X) # Forward process
loss = criterion(logits, labels) # CE loss
loss.backward()
# Calculate gradient
optimizer.step()
# Use gradient to update
```

Multinomial Logistic Regression: Pseudocode

Goal: estimate (W, b) by minimizing the multiclass cross-entropy loss.

1 Initialize parameters

Choose initial values for W and b (e.g., zeros or small random values).

$$W^{(0)}, \quad b^{(0)}$$

2 Forward computation

For each sample $i = 1, \ldots, n$, compute logits and softmax probabilities.

$$s_i = W^\top x_i + b$$

$$p_{ik} = \frac{e^{s_{ik}}}{\sum_{j=1}^{K} e^{s_{ij}}}, \qquad k = 1, \ldots, K$$

3 Compute loss

Evaluate the average multiclass cross-entropy.

$$\mathcal{L}(W, b) = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} y_{ik} \log p_{ik}$$

4 Compute gradients

Differentiate the loss with respect to W and b.

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{1}{n} \sum_{i=1}^{n} x_i (p_i - y_i)^\top \qquad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n} \sum_{i=1}^{n} (p_i - y_i)$$

Here p_i and y_i are K-dimensional vectors.

5 Update parameters

Use gradient descent with learning rate α.

$$W \leftarrow W - \alpha \frac{\partial \mathcal{L}}{\partial W} \qquad b \leftarrow b - \alpha \frac{\partial \mathcal{L}}{\partial b}$$

6 Check stopping criterion

Stop if the loss changes very little or the maximum number of iterations is reached; otherwise repeat from Step 2.

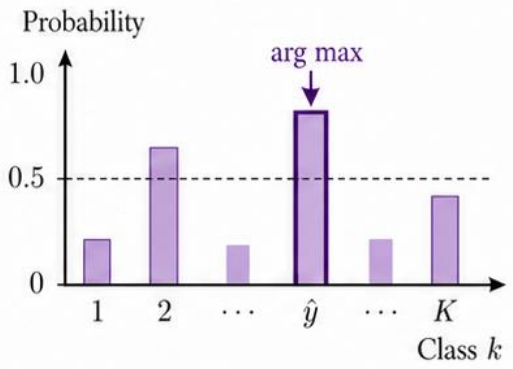
Prediction

For a new input x_{new}:

$$s_{\text{new}} = W^\top x_{\text{new}} + b$$

$$p_{\text{new},k} = \frac{e^{s_{\text{new},k}}}{\sum_{j=1}^{K} e^{s_{\text{new},j}}}, \quad k = 1, \ldots, K$$

$$\hat{y} = \arg\max_k \, p_{\text{new},k}$$

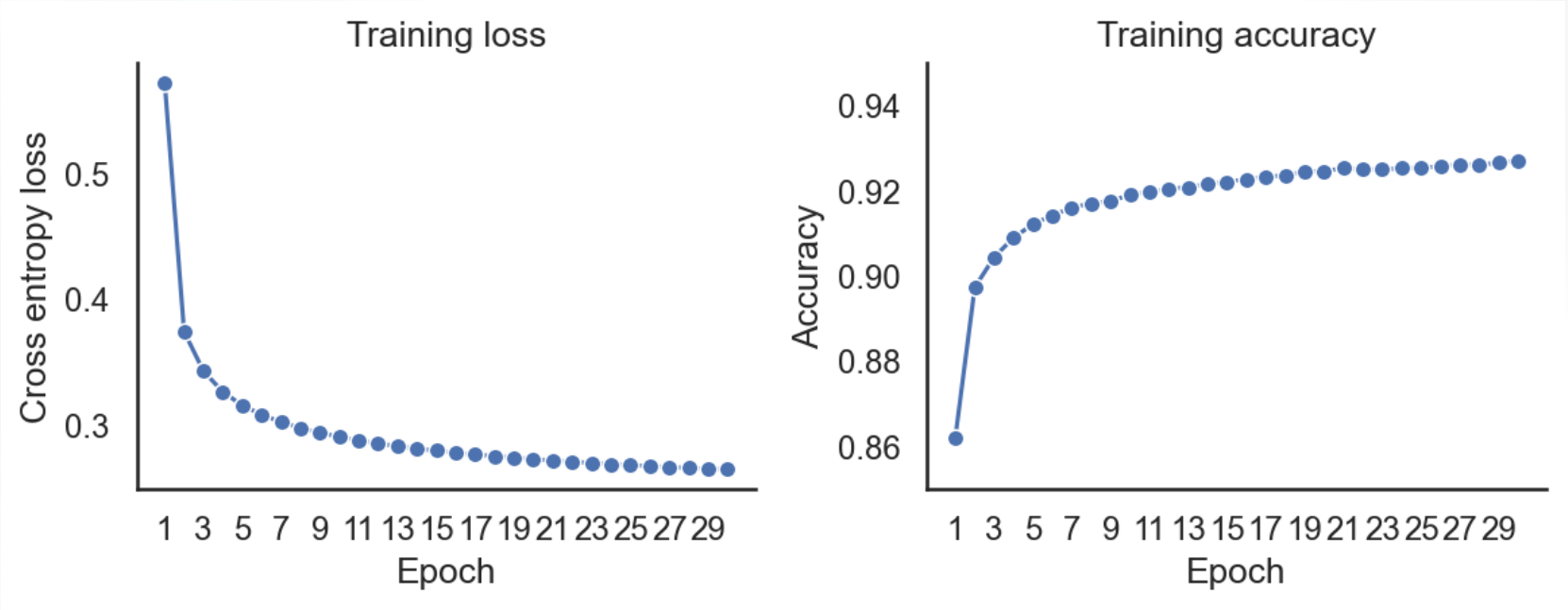


Notation

x_i: feature vector; y_i: class label; y_{ik}: one-hot target; s_{ik}: logit for class k; p_{ik}: probability of class k; α: learning rate

Results

- Test accuracy: 0.9222 with 30 epochs (~30s on a Macbook M4 chip)
- Try by yourself!



Code: dl_intro/03_logistic_regression_mnist_pytorch.ipynb

Recap: from image to prediction

1 Task



Input: a 28×28 image

Goal: predict one label in $\{0, \ldots, 9\}$

2 Linear model



$$s = Wx + b$$

10 raw scores (one per class)

3 Softmax



$$p = \mathrm{softmax}(s)$$

convert scores into probabilities

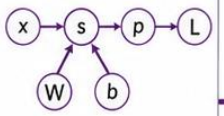
4 Cross-entropy loss



$$L = -\sum_k y_k \log p_k = -\log(p_{\text{correct}})$$

low loss if the correct class gets high probability

5 Backpropagation



$$\frac{\partial L}{\partial s} = p - y$$
$$\frac{\partial L}{\partial W} = (p - y)\, x^\top$$
$$\frac{\partial L}{\partial b} = p - y$$

chain rule

6 SGD update



$$\theta \leftarrow \theta - \eta \nabla_\theta L$$

use gradients to reduce loss

PyTorch

```
logits = model(x)
loss = criterion(logits, y)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

- forward → loss → backward → update
- `CrossEntropyLoss` takes logits

Notation

- $x \in \mathbb{R}^{784}$ (flattened 28×28 image)
- $W \in \mathbb{R}^{10 \times 784}$, $b \in \mathbb{R}^{10}$
- $s \in \mathbb{R}^{10}$ (scores)
- $p \in \mathbb{R}^{10}$ (probabilities)
- $y \in \{0, 1\}^{10}$ (one-hot label)
- $\theta = \{W, b\}$ (parameters)



Core idea: the model predicts, the loss measures error, backprop computes gradients, and SGD updates parameters.

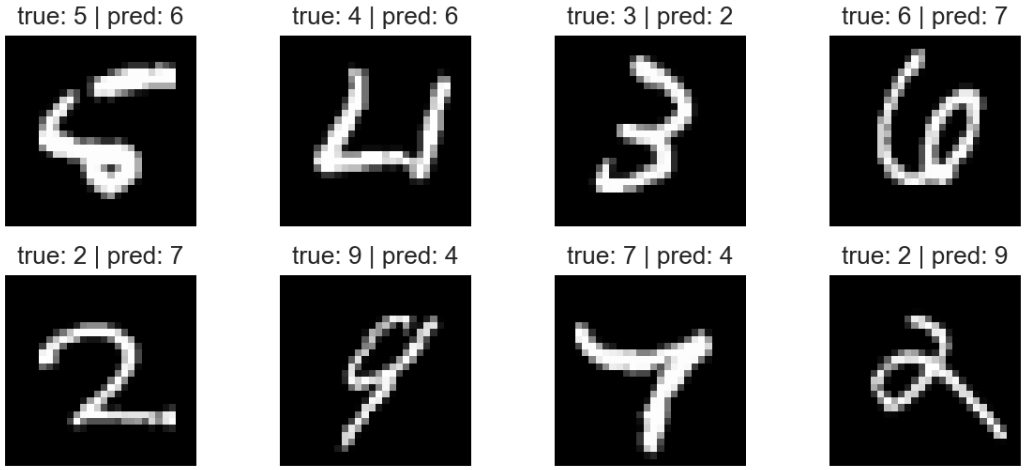
Logistic Regression with MLP

Limitation of linear model

- linear model already works reasonably well, but its errors reveal a limitation: it only uses additive first-order pixel evidence and cannot learn nonlinear feature combinations.
- For each class:

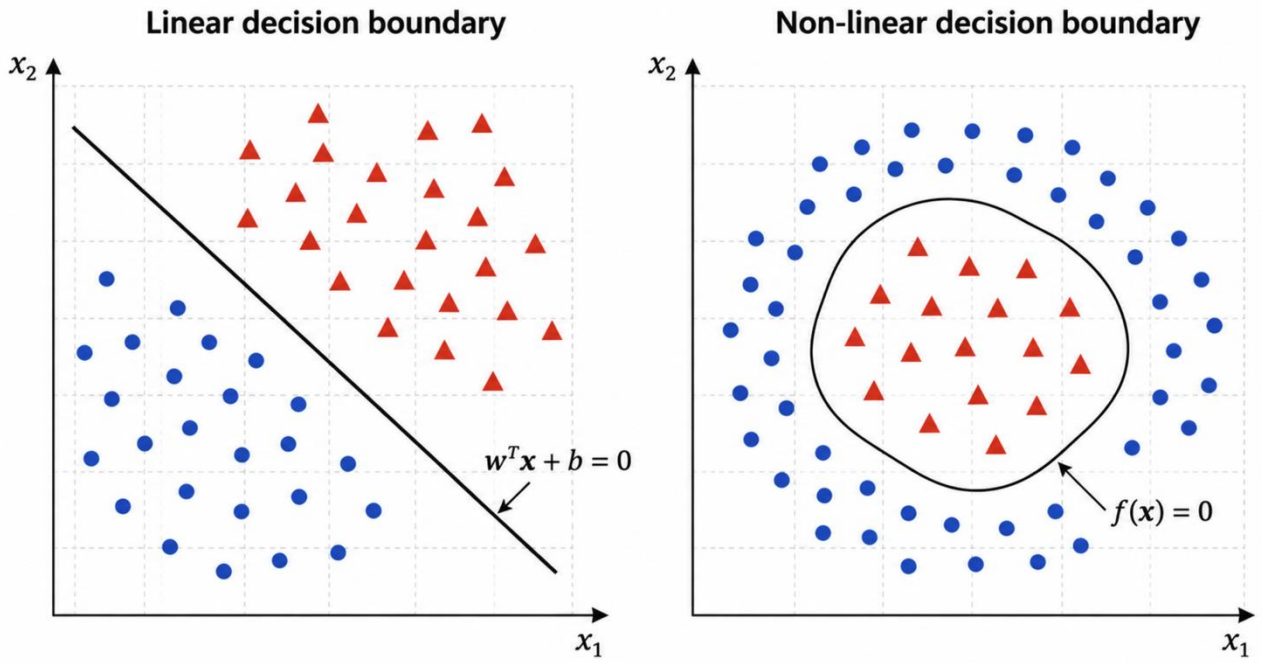
$$s_k = \sum_{j=1}^{784} W_{kj} x_j + b_k$$

- The score is a weighted sum of individual pixels.
- It uses **first-order pixel evidence** only.
- It does not explicitly model interactions such as: $x_i x_j, \quad x_i x_j x_l$



Why nonlinearity?

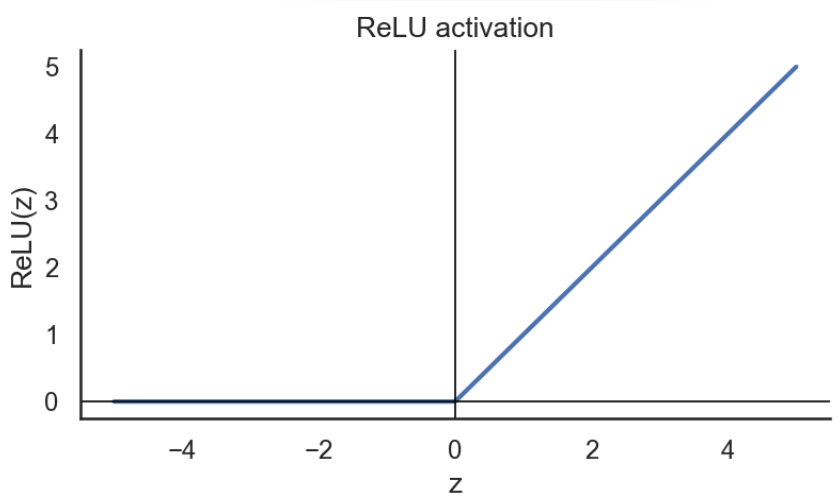
- A linear model creates linear decision boundaries in feature space.
- Some patterns cannot be separated well by a single linear rule.
- Nonlinear models can bend the decision boundary.
- More importantly, they can learn intermediate features: pixels -> strokes -> digits



Introduce nonlinearity: ReLU

- ReLU means Rectified Linear Unit.
- It keeps positive values and removes negative values:

$$\mathrm{ReLU}(z) = \max(0, z)$$



- Linear + Linear = Still Linear
- Linear + ReLU + Linear = Nonlinear model

- Of course, you can use other non-linear functions, e.g. Sigmoid, SiLU, tanh

From linear model to MLP

- Linear classifier: $x \longrightarrow s$
- MLP(Multi Layer Perceptrons) adds a hidden layer:

$$h = \mathrm{ReLU}(W_1 x + b_1)$$

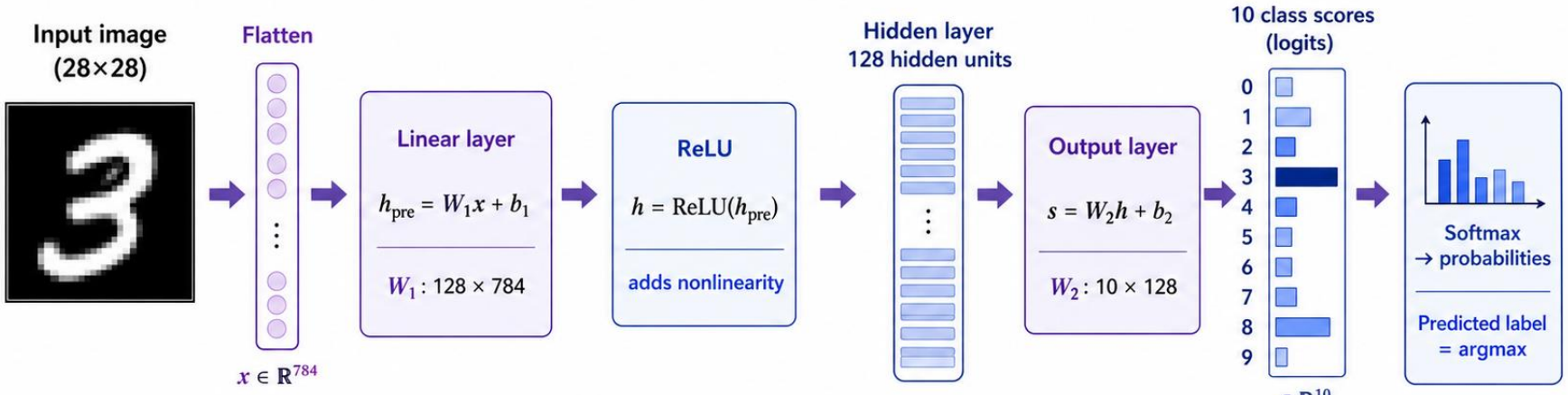
$$s = W_2 h + b_2$$

- The hidden layer learns intermediate features.
- The output layer uses these features for classification.

- Linear + activation + Linear = MLP!
- Can have many layers. Must have an activation between two layers.

- Gradient? Pytorch automatically do it for you.

2 Layer MLP



1. Layer 1 learns useful intermediate features from pixels

2. ReLU lets the model learn nonlinear patterns

3. Layer 2 maps hidden features to 10 digit classes

Overall (forward pass) equations

$$h = \mathrm{ReLU}(W_1 x + b_1) \qquad s = W_2 h + b_2$$

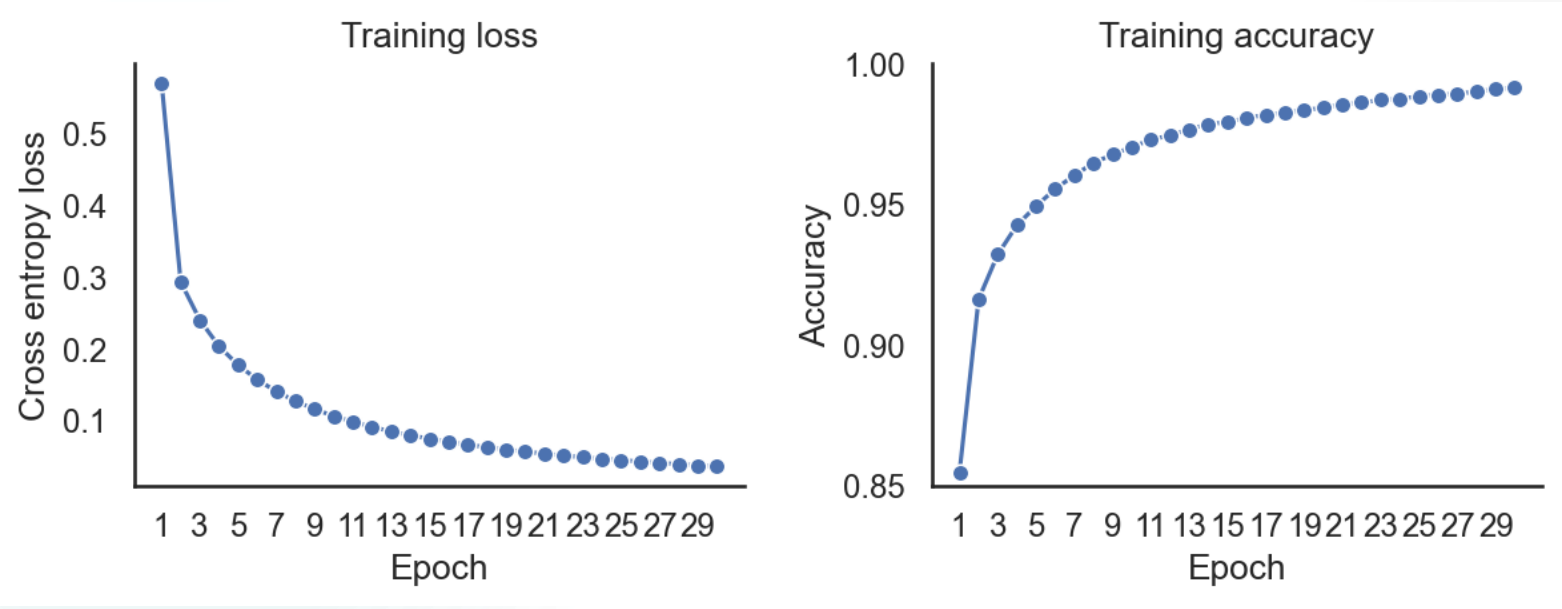
- $x \in \mathbb{R}^{784}$: input vector (flattened 28×28 image)
- $h \in \mathbb{R}^{128}$: hidden representation
- $s \in \mathbb{R}^{10}$: output scores (logits)
- W_1, b_1, W_2, b_2 : learnable parameters

During training, we minimize a loss (e.g., cross-entropy) between the softmax probabilities and the true label.

```python
model = nn.Sequential(
    nn.Linear(28 * 28, HIDDEN_SIZE),
    nn.ReLU(),
    nn.Linear(HIDDEN_SIZE, 10),
).to(device)
```

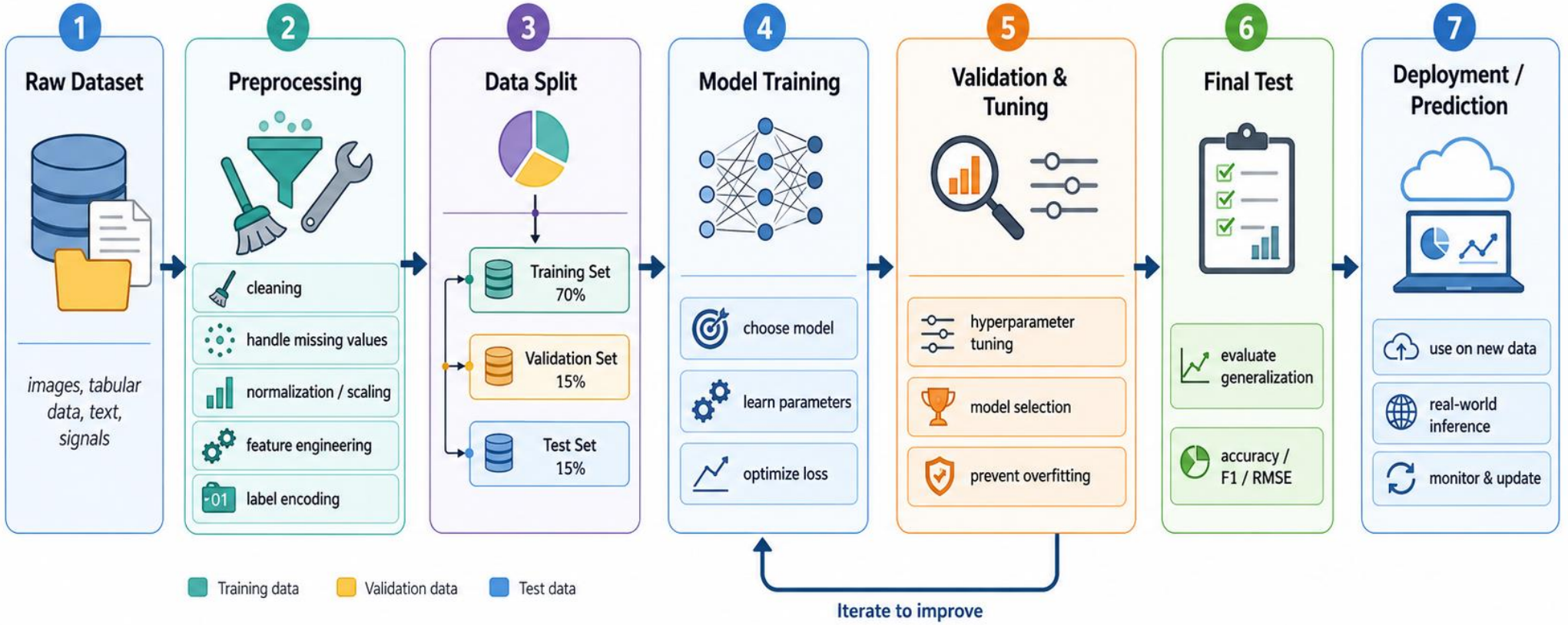
Performance

- Test set accuracy: 0.9770



Code: dl_intro/04_mlp_mnist_pytorch.ipynb

Recap: a standard pipeline of machine learning



From raw data to evaluation and real-world prediction

After class

- Run all code by yourself !!
- The only way to learn deep learning is read, understand, execute, modify the code by yourself.
- It’s very interesting to tune the hyper-parameters and get a good result.

- The fifth notebook talks about overfitting, which we didn’t mention in our class, but it’s very important.
- The sixth notebook guide you to actually understand a Pytorch code.
- If you want to learn more, follow guidance in README.md or ask TA.